

Course: ENSF 460 – Embedded Software and Hardware Systems

Project #: Project 1 (Count Down Timer App)

Instructor Name: Benjamin Tan

Student Name(s): Stephen Ravelo, Aaron Lauang, Alexa González

Lab Section: B02

Date submitted: October 26, 2025

Peripheral Initialization and Setup

Peripherals and setup are done at the beginning of function main.

```
int main(void)
{
    AD1PCFG = 0xFFFF; /* keep this line as it sets I/O pins that can also
    be analog to be digital */

    newClk(500); // Set clock frequency to 500kHz

    //Timer config
    timerInit();

    /* Let's set up some I/O */
    I0init();

    /* Let's clear some flags */
    CN_event = 0;

    IPC4bits.CNIP = 6;
    IFS1bits.CNIF = 0;
    IEC1bits.CNIE = 1;

    /* Let's set up our UART */
    InitUART2();

    Disp2String("\033[2J\033[H\r"); // clear terminal
    ...
}
```

timerInit() is found in defined in timer.c. The function configures timer 2 to our desired parameters.

timer.c

```
void timerInit() {
    // TMR2 config
    T2CONbits.T32 = 0;
    T2CONbits.TCKPS = 3;    // set prescalar to 256
    T2CONbits.TCS = 0;
    T2CONbits.TGATE = 0;
    T2CONbits.TSIDL = 0;
    IPC1bits.T2IP = 2;
    IFS0bits.T2IF = 0;
    IEC0bits.T2IE = 1;
}

void delay_ms(uint16_t time_ms) {
    PR2 = 1 * time_ms;    // PR2 coefficient: 1 ~= 0.001 * 250000 / 256
    TMR2 = 0;

    T2CONbits.TON = 1;

    // idle until timer 2 interrupt
    while (T2CONbits.TON == 1) {
        Idle();
    }

    return;
}
```

Using a pre-scaler of 256 and clock frequency 500 kHz, our PR2 calculation comes in to about 2.3% off 1. Using this value allows us to be accurate for most delays and utilize create delays up to the capacity of an unsigned 16-bit integer (65535 ms).

We then call `IOinit()` from `IOs.c`. The function configures our pin I/O.

`IOs.c`

```
uint16_t CLR_F = 0;

// Initialize peripheral IO
void IOinit() {
    // LED1
    TRISBbits.TRISB9 = 0;

    // LED2
    TRISAbits.TRISA6 = 0;

    // PB1
    TRISBbits.TRISB7 = 1;
    CNPU2bits.CN23PUE = 1;
    CNEN2bits.CN23IE = 1;

    // PB3
    TRISBbits.TRISB4 = 1;
    CNPU1bits.CN1PUE = 1;
    CNEN1bits.CN1IE = 1;

    // PB3
    TRISAbits.TRISA4 = 1;
    CNPU1bits.CN0PUE = 1;
    CNEN1bits.CN0IE = 1;
}
```

Interrupts are enabled for each of the buttons. Both LEDs will also be used in this project.

Program Logic

Our logic occurs in the main while(1) loop.

main.c

```
int main(void)
{
...

    // Main Loop
    // Waits for button press and executes logic after 200ms
    while(1)
    {
        Idle();           // Wait for interrupt
        delay_ms(200);     // Debouncing
        if(CN_event == 1)
        {
            CN_event = 0;
            IOcheck();      // Execute IO Logic

            _LATB9 = 0;     // Turn off LED
        }
    }

    return 0;
}
```

Every iteration, the CPU remains in idle until an interrupt occurs. To account for multi-button presses, we use TMR2 to delay for 200ms before executing any logic. CN_event is set to 1 whenever an interrupt occurs, which then calls IOcheck() in IOs.c.

main.c

```
// CN interrupt subroutine
void __attribute__((interrupt, no_auto_psv)) _CNInterrupt(void)
{
    IFS1bits.CNIF = 0; // Clear CN interrupt flag

    // Only trigger CN_event if a PB is pressed
    if(PORTBbits.RB7 == 0 || PORTBbits.RB4 == 0 || PORTAbits.RA4 == 0){
        CN_event = 1;
    }
}
```

IOs.c

```
// Execute Logic for peripheral IO
void IOcheck() {
    uint16_t count = 0; // used for inputs that include a short/long
                        // press function

    // PB1 pressed: increment seconds by 1 and display set message
    // After incrementing 5 seconds and seconds value ends in 0 or 5
    // start incrementing by 5 seconds.
    if (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 1 && PORTAbits.RA4 == 1) {
        while (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 1 &&
            PORTAbits.RA4 == 1) {
            if (count >= 5 && getSeconds() % 5 == 0) {
                incrementSeconds(5);
            } else {
                incrementSeconds(1);
                count++;
            }
            displaySET();
            delay_ms(500);
        }
        count = 0;
    }
    // PB2 pressed: increment minutes by 1 and display set message
    else if (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 0 &&
        PORTAbits.RA4 == 1) {
        while (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 0 &&
            PORTAbits.RA4 == 1) {
            incrementMinutes(1);
            displaySET();
            delay_ms(500);
        }
    }
}
```

```

// PB1 and PB2 pressed:
// Short press (< 3 seconds): start timer (must not be 00:00)
// Long press (> 3 seconds): reset timer and display set message
else if (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 1) {
    while (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 1) {
        if (count == 60) {
// reset timer if PB1 and PB2 held for 500 ms
            resetTimer();
            displaySET();
        }
        delay_ms(50);
        count++;
    }

    // start timer if held for 3 seconds and seconds or minutes > 0
    if(count < 60 && (getSeconds() || getMinutes())) {
        startTimer();
        if (!CLRF) {
// if timer has not been cleared while running
// trigger alarm at end of timer
            alarm();
        }
        CLRF = 0;
    }
    count = 0;
// PB1 and PB3 pressed: decrement seconds by 1 and display set message
else if (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 1 &&
PORTAbits.RA4 == 0) {
    while (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 1 &&
PORTAbits.RA4 == 0) {
        decrementSeconds(1);
        displaySET();
        delay_ms(500);
    }
}
}

```

```

// PB2 and PB3 pressed: decrement minutes by 1 and display set message
else if (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 0) {
    while (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 0) {
        decrementMinutes(1);
        displaySET();
        delay_ms(500);
    }
}
// ALL PBs held down: display group info on terminal
else if (PORTBbits.RB7 == 0 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 0) {
    uint16_t groupInfoDisplayed = 1;
    while(PORTBbits.RB7 == 0 && PORTBbits.RB4 == 0 &&
PORTAbits.RA4 == 0) {
        if(groupInfoDisplayed == 1) {
            displayGroupInfo();
            groupInfoDisplayed = 0;
        }
    }
    displaySET();
}
}

```

Functions of each listed button press are described in the code comments. Key takeaways include the use of a *count* variable, which we use in combination with *void delay_ms()* from *timer.c* to achieve short/long press functions. In the case of the timer having a value of 00:00, the timer cannot be started while in its “not running” state.

Once the timer starts (short press of PB1 and PB2, < 3 seconds), the program runs in `void startTimer()` in *ProgramTimer.c*.

ProgramTimer.c

```
// Will run timer until seconds and minutes reach zero
void startTimer() {
    while ((minutes > 0 || seconds > 0) && paused == 0) {
        if (seconds == 0) {
            minutes -= 1;
            seconds = 59;
        } else {
            seconds -= 1;
        }
        _LATB9 ^= 1;    // Toggle LED1 every second
        displayCNT();
        delay_ms(910);
        IOcheckRunning();
    }
    _LATB9 = 0;
}
```

Using a delay of 910 ms, the function accounts for the overhead of other instructions and allows the timer to be nearly perfect when compared to another timer. This value of 910 ms was determined experimentally and could be much more accurate if a low-level instruction trace was used to determine the exact number of instructions that occurred. Knowing the instruction count would allow us to calculate the approximate time that has passed between each delay, which could be subtracted from 1000 ms to determine the perfect number for `void delay_ms()`.

If PB3 is pressed, we escape out of the delay and check for IO specific to when the timer is running using `void IOcheckRunning()` in `IOs.c`.

IOs.c

```
// IO check for while the timer is running
void IOcheckRunning() {
    uint16_t count = 0;

    // PB3 pressed:
    // Short press (< 3 seconds): pause timer
    // Long press (> 3 seconds): reset timer and display clear message
    if (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 1 && PORTAbits.RA4 == 0) {
        PB3_event = 1;
        while (PORTBbits.RB7 == 1 && PORTBbits.RB4 == 1 &&
            PORTAbits.RA4 == 0) {
            if (count == 60) {
                // reset timer after PB3 is held for 3 seconds
                // while timer is running
                resetTimer();
                displayCLR();
                CLRf = 1;
            }
            delay_ms(50);
            count++;
        }
        if (count < 60) { // otherwise, pause the timer
            pauseTimer();
        }
        count = 0;
        PB3_event = 0;
    }
}
```

The function works similarly to `void IOcheck()`, but it only checks for PB3. The use of this function is as described in the comments, however, notice the CLRf (clear flag) variable is set to 1. This variable prevents the alarm from getting set off when it is reset while the timer is running.

Other notable features

`void delay_ms(uint16_t time_ms)` in `timer.c` protects from CN interrupts. The function does this by only returning from `Idle()` after timer 2 has completed.

`timer.c`

```
void delay_ms(uint16_t time_ms) {
    PR2 = 1 * time_ms;      // PR2 coefficient: 1 ~= 0.001 * 250000 / 256
    TMR2 = 0;

    T2CONbits.TON = 1;

    // idle until timer 2 interrupt
    uint16_t skip = 0;
    while (T2CONbits.TON == 1 && !skip) {
        Idle();
        if (!getPB3_event() && PORTAbits.RA4 == 0) {
            skip = 1;
        }
    }

    return;
}
```

For the specific event in which we check for PB3 while the timer is running, we use an event flag to escape out of the initial delay in `void startTimer()` in `ProgramTimer.c` so the response to the inputs would be instant.

`void Disp2Dec(uint16_t)` in `UART2.c` was modified to fit the needs of the assignment. The counter (ctr) was changed to only display 2 digits when called, and we removed the space that the function would add before each number displayed.

```
// Displays 16 bit unsigned in decimal form
void Disp2Dec(uint16_t Dec_num)
{
    uint8_t rem; //remainder in div by 10
    uint16_t quot;
    uint8_t ctr = 0; //counter
    while(ctr<2)
    {
        quot = Dec_num/(pow(10,(1-ctr)));
        rem = quot%10;
        XmitUART2(rem + 0x30 , 1);
        ctr = ctr + 1;
    }
    // XmitUART2('\n',1); // new line
    // XmitUART2('\r',1); // carriage return

    return;
}
```

State Machine Diagram

